

E7 Spring 2026

Midterm practice problem set #2

(version 1)

Instructions

- Multiple choice questions with circular bullets have a single correct answer, and you should select only one option.
- Multiple choice questions with square bullets may have more than one correct answer, and you should select all that apply.
- The following package imports apply to all problems.

```
import math
import cmath
import numpy as np
import pickle
from pathlib import Path
```

Week 5: Strings

1) General questions

1. Are strings a scalar or a collection data type?
2. Are strings ordered?
3. Are strings mutable?
4. Which Python string method accomplishes each of these tasks (`strA`, `strB`, `strX`, and `strY` are strings).
 - (a) Check whether `strA` starts with `strB`.
 - (b) Check whether `strA` ends with `strB`.
 - (c) Find the first occurrence of `strB` within `strA`.
 - (d) Count the occurrences of `strB` within `strA`.
 - (e) Replace the occurrences of `strX` within `strA` with `strY`.
 - (f) Convert the letters in `strA` to lower case.
 - (g) Remove leading and trailing whitespace from `strA`.
 - (h) Divide `strA` into a list of strings at a given delimiter `delim`.
 - (i) Join a list of strings into a single string using a delimiter `delim`.
5. What happens when you add two strings `strA+strB`?
6. What happens when you multiple a string by an integer?

2) String methods

2.1) What is the value of `result` after executing this code?

```
data = "  2024-02-28, 101.5, kPa  "  
clean_data = data.strip().replace("kPa", "bar")  
parts = clean_data.split(", ")  
result = ":".join(parts)
```

2.2) What are the values of `check1` and `check2` after executing this code?

```
sensor_id = "SN-405-Alpha"  
check1 = sensor_id.startswith("sn")  
check2 = sensor_id.lower().startswith("sn")
```

2.3) What is the value of `modified` after executing this code?

```
coords = ["12.5", "45.0", "110.2"]  
csv_line = ",".join(coords)  
modified = csv_line.replace(".", ":")
```

2.4) The following string with comma-separated data is given.

```
data = "10.1,20.5,30.9"
```

Which of these lines of code produces the same string but with a semicolon instead of a comma delimiter.

- ☐ `data.split(",").join(";")`
- ☐ `data.find(',').replace(',')`
- ☐ `",".join(data.split(","))`
- ☐ `data.replace(",", ";")`

2.5) A configuration file contains the line

```
setting = "Frequency : 60Hz"
```

Which code snippet correctly extracts the integer 60 as a string?

- ☐ `setting.find("60")`
- ☐ `setting.get(60)`
- ☐ `setting.split(":")[1].strip().replace("Hz", "")`
- ☐ `setting.split(":")[0].strip("Hz")`

2.6) Given the definition of `headers`:

```
headers = ["Time", "Voltage", "Current"]
```

Which of the following lines of code produce a single string where headers are separated by a semicolon?

- ☐ `";" .split(headers)`
- ☐ `headers.join(";")`
- ☐ `";" .join(headers)`
- ☐ `headers.replace(" ", ";")`

2.7) Suppose you have a data file where missing values are marked with either `--` or `NaN`. A single line from this file is stored in the variable `line`. For example,

```
line = "25.2, --, 26.1, --, NaN, 4.3"
```

Which of the following lines of code correctly counts the number of missing values?

- ☐ `line.replace("--", "NaN").count("NaN")`
- ☐ `line.find("--", "NaN").count("NaN")`
- ☐ `line.replace("NaN", "--").count("--")`
- ☐ `line.replace("--", "NaN").find("NaN")`

2.8) You are reading a file where the header line might be labeled as `"Time"`, `"time"`, or `"TIME"`. Which of the following expressions should be used to check that the header is correctly formatted?

- ☐ `line.startswith("Time") or line.startswith("time")`
- ☐ `line.lower().startswith("time")`
- ☐ `line.find("time") == 0`
- ☐ `line.split()[0] == "time"`

3) f-strings

3.1) Use an f-string to display the gravitational constant `G` rounded to exactly 3 decimal places in scientific notation.

3.2) Use an f-string to create a table header with the word `"Data"` left-aligned in a 10-character wide block, with dashes `(-)` filling the empty space.

3.3) Use an f-string to display the integer `iters` using commas as thousands separators.

3.4) Use an f-string to display the float `v` using 2-decimal precision and right-aligned within a string of length 8, and filling the empty space with spaces.

```
v = 12.5
```

Week 6: Files and Modules

4) General questions

1. What is the “home” directory?
2. What is the “current working directory”?

5) Paths

5.1) Which of the following define a path to a file called `data.csv`, located in a folder called `results` in the current working directory?

- ☐ `Path.cwd() + "results" + "data.csv"`
- ☐ `Path.cwd() / "results" / "data.csv"`
- ☐ `Path.cwd()/results/data.csv`
- ☐ `Path.cwd()/results/data.csv"`

5.2) Suppose we have a `Path` object called `mainfile` that references a file called `main.c`. Inspecting `mainfile` we find,

```
print(mainfile)
```

```
/home/chad/mydata/main.c
```

Which of the following address the user’s home folder (`/home/chad`)?

- ☐ `Path.home()`
- ☐ `mainfile.home`
- ☐ `mainfile.parents[1]`
- ☐ `mainfile.parent.parent`

5.3) Which of the following expressions correctly check that the path `p` exists and that it is a directory?

- ☐ `p.exists().is_dir()`
- ☐ `p.exists() and not p.is_file()`
- ☐ `p.exists() and p.is_dir()`
- ☐ `p.exists() or p.is_dir()`

5.4) We wish to create a new directory called `logs` under the current working directory, and then create an empty file called `session.txt` inside it. Choose the code(s) that correctly accomplish this task.

☐

```
logspath = Path.cwd()/"logs"  
filepath = logspath/"session.txt"  
if not logspath.exists():  
    logspath.mkdir()  
filepath.touch()
```

☐

```
logspath = Path.cwd()/"logs"  
filepath = logspath/"session.txt"  
if logspath.exists():  
    logspath.touch()  
filepath.mkfile()
```

☐

```
logspath = Path.cwd()/"logs"  
filepath = logspath/"session.txt"  
if not logspath.exists():  
    logspath.touch()  
filepath.mkfile()
```

☐

```
logspath = Path.cwd()/"logs"  
filepath = logspath/"session.txt"  
if logspath.exists():  
    logspath.mkdir()  
filepath.touch()
```

5.5) Write a list comprehension that collects oaths for all of the text files (i.e. with extension `".txt"`) in your home folder.

5.6) Which of the following lines of code produce the string "84723"?

```
p = Path.home()/'data'/'84723.txt'
```

- ☐ `int(p.name)`
- ☐ `p.name - p.suffix`
- ☐ `p.name[:-4]`
- ☐ `p.name.split(".")[0]`

5.7) Which of the following will create a path to an existing directory regardless of the operating system on which it is run?

- ☐ `Path.cwd()`
- ☐ `Path.root()`
- ☐ `Path.home()`
- ☐ `Path("home")`

6) Namespaces and name shadowing

6.1) What does the following code print?

```
x = 50

def my_function():
    x = 10
    print(x)

my_function()
print(x)
```

- ☐ 50 then 50
- ☐ 10 then 10
- ☐ 10 then 50
- ☐ 50 then 10

6.2) What does the following code print?

```
name = "Alice"

def f1():
    global name
    name = "Bob"
```



```
def f2():  
    name = "Charlie"  
  
f1()  
f2()  
print(name)
```

- ☐ Alice
- ☐ Bob
- ☐ Charlie
- ☐ An empty string.

6.3) What does the following code print?

```
A = [0,0,0]  
print = A  
print(A)
```

- ☐ [0,0,0]
- ☐ 0
- ☐ Nothing, it raises an error.

6.4) What is the final value of `val` after this script runs?

```
val = 5  
  
def f1():  
    global val  
    val *= 2  
  
def f2():  
    val = 20  
    return val  
  
f1()  
f2()
```